AMIT MALHOTRA

Forward Deployed AI Engineer · Agent Reliability · LLM Gateways · Private AI · Context Engineering

Hicksville, NY · amit.malhotra1@gmail.com · (917) 345-7516 · github.com/amit-malhotra1 Available for full-time, contract and 1099 engagements.

PROFILE

I build the infrastructure that makes AI agents dependable in real work. I bring **21 years of enterprise data and risk technology**, including Moody's Analytics and Basel/CCAR platforms for major banks, and, since 2025, full-time hands-on engineering on LLM-agent systems (20+ shipped tools and platforms). Over that time I moved past early LLM experimentation to the problems underneath it:

- **Vendor-independent model access.** I built my own LLM gateway that runs the Claude Code agent against models on Google Vertex AI, AWS Bedrock, OpenRouter or local hardware. It needs no Claude subscription and keeps prompt caching and tool calling working.
- **Private AI on my own hardware.** Ollama and Open WebUI on a 64 GB Apple Silicon workstation, running 30B-class models with long context windows, and my own chat assistant that combines local models with cloud APIs.
- **Agents that can be trusted.** I built a drop-in governance standard that makes any AI coding agent resume where it stopped, back up before it edits, stay in scope and prove its work with measured tests.
- **Context as an engineered pipeline.** Capture → compress → compile → route → cache → verify, built and rebuilt over a year of iterations.
- **Deterministic computation with LLM interpretation.** Math and rules produce the answer; the model only explains it.

I like taking a complex process, understanding it thoroughly, breaking it into small pieces and rebuilding it so it is simple to use. Everything I build is **lightweight, portable and plug-and-play**, designed from the start to connect with the other tools I've built.

I work the way a forward-deployed engineer does. I go into a team's real workflow, find where the AI fails (hallucinated "done", lost work, runaway cost, context collapse, vendor lock-in), and ship the harness that fixes it.

WHAT I DELIVER

Engagement	What I build	Typical outcome
AI coding-agent rollout & governance	Hooks, plan-before-edit workflow, automatic backups, scope guards, append-only audit records, measured-evidence reporting for Claude Code / Codex / Gemini	Agents that resume, can't silently destroy work, and leave an audit trail a reviewer or regulator can read
LLM gateway & cost control	Local or self-hosted proxy: provider routing, schema translation, prompt-cache injection, per-request cost telemetry, failover	Lower token spend, no single-vendor lock-in, freedom to choose the model per task
Private / on-prem AI	Local model hosting (Ollama), a self-hosted chat front end (Open WebUI), private web search, multi-machine setups, hardware and model sizing	AI capability for data that can't leave the building, at near-zero marginal cost
Custom MCP servers	Tool servers that connect agents to code, documents and internal systems, with pass-by-path loading, anti-truncation and drill-down storage	Agents get deep analysis without context bloat or truncated answers
Deterministic AI pipelines	"Compute, then narrate" engines: calculations, scoring and validation in code; the LLM gets schema-validated evidence	Reproducible results for regulated or high-stakes domains
Vision / computer-use agents	VLM perception → deterministic policy table → verified action, with session token budgeting	Automation of screens and legacy apps that have no API
Integration & simplification	Breaking a tangled workflow into small, reusable modules that plug into each other	One clean system in place of many disconnected tools and scripts

HOW I THINK AND WORK

Understand deeply, then simplify

When a problem is worth solving and the reasoning behind it is strong, I get into the details. I map the whole process, find where it really breaks, then **break it into small, testable pieces** and rebuild it so it is easier to use. The goal is always the same: the complexity sits inside the system, and the person using it gets a simple surface: one command, one menu, one health check.

Lego architecture: flexible, lightweight, portable, plug-and-play

Every tool is built as a **self-contained module**: one folder or one file, few or no dependencies, its own README, an installer or deploy script. That way it can be dropped into another project and combined with other modules. While building any single tool, I am already thinking about how it will **connect and stay in sync** with the rest of the ecosystem (see *The Connected Ecosystem* below).

- **Skills as building blocks.** Large instruction sets are broken into atomic skills. A complex skill is split into numbered packets that a master skill unpacks in sequence.
- **Composed workflows.** Named workflows are assembled from atomic skills; for example *safe build* = core rules + guard + feature + test + diff, and *safe cleanup* = audit + guard + refactor + docs.
- **Swappable components.** Strategy "cartridges" (in the vision agent and the skill viewer) can be exchanged without touching the engine.
- **Skill discovery by code.** The AMTrix Plan standard reads a project's real commands (package scripts, Makefile targets, docker compose) and suggests the skills that project should have.

Visual-first learning

I learn fast by making systems visible, and I keep that knowledge by documenting it visually:

- **Mermaid architecture and sequence diagrams** in nearly every repository.
- **A diagram generator I built**, which renders Mermaid dashboards from structured text (SVG output, tables embedded in diagram nodes, 15+ iterations).
- **Generated diagram pages** in the AMTrix Plan board.
- **Consistent icon and color status markers** in documentation and terminal dashboards, so a page or a HUD can be read in seconds.

Seeing a system is how I understand it, and it is how I make it understandable to teams and clients.

Engineering principles

1. **Determinism over cleverness.** Priority lookup tables instead of free-form reasoning. Byte-exact, hashed request payloads. Every response classified as complete, partial, failed or unknown.
2. **Nothing is ever lost.** Backup before the first edit, versioned sibling files instead of overwrites, append-only locked logs, anti-truncation gates.
3. **Measured, not claimed.** Test counts and timings are recorded by code, never typed by a model.
4. **Context is a budget.** Pass-by-path loading, compiled context handoffs, redundancy compression, cache TTL management, per-screenshot token accounting.
5. **Model-agnostic.** Claude, Gemini, GPT, Kimi K2, DeepSeek, Grok, Llama and Qwen: the right model per task, behind one interface.
6. **Operator-first.** The system carries the memory, the checking and the routine, so the human doesn't have to.

THE CONNECTED ECOSYSTEM

The projects below are not isolated experiments. They are modules that feed and govern each other.

Layer	Module	Feeds or governs	How
Model access	**Claude Code Center** (LLM gateway)	Private AI Platform	Routes its $0 local tier to the Ollama models

Layer	Module	Feeds or governs	How
Model access	**Private AI Platform** (Ollama + Open WebUI)	AMTrix Studio, AI Companion, the gateway	One local model tier shared by every tool
Governance	**AMTrix Plan** standard	Frontier Triage Engine	The engine was born inside the standard and spun out
Governance	**AMTrix Plan** standard	AMTrix Core, Availability Platform, this portfolio	Installed governance record in each; one command updates all
Governance	**Frontier Triage Engine**	Universal MCP Server	Embedded for hashed, audited model calls
Governance	**Patch TUI**	Claude Code Center	Shipped a multi-file gateway release as fingerprinted patches
Context	**Agent Protocol / AI Protocol Template**	Every active project	On-disk memory and rules installed and synchronised by a fleet updater
Context	**AMTrix Core** (skills + compilers)	Claude Code Center, Vision agent, Mac/iPhone	QA skill drives the gateway's diagnostics script; skills compile to editor snippets and text replacements
Context	**Context Compressor**	Agent Protocol	Bundled inside the protocol
Signal	**VoiceBus + Notify-Bus**	Availability Platform, long-running agents	Alerts and agent milestones over one self-hosted push contract
Applications	**Availability Platform**, **Vision Decision Agent**, **Ephemeris Engine**	(consumers)	Run under the governance, context and signal layers above

Shared module	Where it is reused
AI Protocol Template (agent rules + on-disk memory)	Installed in every active project, including the AI Companion Toolkit, Private AI Platform, AMTrix Studio and the Availability Platform
AMTrix Plan standard	AMTrix Core, the Availability Platform and other projects; one command updates every installed project
Frontier Triage Engine	Born inside AMTrix Plan, spun out as a standalone engine, then embedded in the Universal MCP Server
Context Compressor	Standalone tool, also bundled inside the Agent Protocol
Local model tier (Ollama)	Open WebUI chat assistant, Claude Code Center's $0 route, AMTrix Studio, AI Companion Toolkit
Signal Layer	Product alerts (Availability Platform) and agent completion/approval notifications
Identity Engine design tokens	A shared visual language across dashboards and web cockpits

---

THE CONTEXT-ENGINEERING JOURNEY (SEP 2025 → OCT 2026)

I didn't take one approach to context engineering. I built seven, measured where each one broke, and kept what survived.

Stage	What I built	What it taught me
1 · Modular instruction design (Sep–Oct 2025)	**PromptForge**: an IDE that treats LLM instruction sets like code (components, includes, conditional assembly, version control)	Structure helps, but it doesn't fix forgetting, drift or hallucination
2 · Persistent memory on disk (Nov–Dec 2025)	**Agent Protocol `.ai` system**: immutable core rules, a living vault (context, session log, friction log), cross-project wisdom and friction indexes, and one-command propagation to every project	Memory must live **on disk** and be **versioned**; the model can't be trusted to hold it
3 · Compression (Jan 2026)	**AI Context Compressor** (Python + Rust): near-duplicate collapse, boilerplate stripping, n-gram shortcode dictionary	Removing redundancy keeps intent intact; summarizing loses it
4 · Routing & traffic inspection (Dec 2025–Mar 2026)	**Agent Protocol `.agent` v17 → v19.5**: a 1,000+-line multi-provider context router, model alias registry, a payload-capture vault that records what AI tools actually send over the wire, a hardened test suite	What a tool **sends** matters more than what it **says** it sends. Control of context belongs at the request layer
5 · Skills & compiled context (Feb–May	**AMTrix Core**: atomic skills, composed workflows, and a Context Compiler that merges spec, task and friction files into one	Small, composable pieces beat one long instruction set

Stage	What I built	What it taught me
2026)	handoff file for any fresh model	
6 · Own the gateway (Mar–Oct 2026)	**Claude Code Center**: my own proxy in front of the agent, with routing, caching, translation and cost telemetry	**The best result:** owning the proxy gives full control of context, cost and model choice, without monthly subscriptions
7 · Govern the process (Oct 2026)	**AMTrix Plan standard** + **Frontier Triage Engine**	Reliability comes from process enforced by code (hooks, backups, evidence), not from better wording

FLAGSHIP WORK

1 · Claude Code Center: Vendor-Independent LLM Gateway for Agentic Coding

Node.js · Express · SQLite · Google Vertex AI · AWS Bedrock · OpenRouter · Ollama · Bash TUI · 97 commits (Mar–Oct 2026)

Problem. The Claude Code agent is one of the strongest coding agents available, but it is tied to Anthropic's own subscription or API. Teams that want to run it on credits in their own cloud account, on other frontier models, or on local models had no off-the-shelf path. Cost per session was also hard to see.

What I built. A local gateway that sits between the Claude Code CLI and every model provider. It handles translation, routing, caching and metering, while the agent keeps working as designed.

Engineering highlights - **Google Vertex AI integration.** The gateway runs the Claude Code harness against models on Vertex AI using the client's own GCP project and pay-per-token billing: - Claude on Vertex through the native `rawPredict` path, with 1-hour prompt caching verified to match the direct API. - Gemini through native Vertex endpoints. - **Third-party Vertex Model-as-a-Service models (Kimi K2, DeepSeek)**: the gateway handles regional OAuth2 and payload translation that the CLI's built-in Anthropic client can't do. - Service-account OAuth2 with token caching, and a model registry that maps friendly names to Vertex model IDs and endpoints. - **When I built it, this was not available in off-the-shelf tooling.** Claude Code supports Claude on Vertex natively; my gateway extends it to Vertex's other model families. - **Tool calling on non-Anthropic models.** A streaming (SSE) accumulator merges fragmented `functionCall` fields and reasoning signatures into clean tool blocks. Claude Code's tool loop therefore runs on Gemini and other models; tool support is flagged per model in the registry. - **Cost engineering:** - Prompt-cache injection with a 1-hour TTL. - Vertex explicit context caching (`cachedContents`) that splits stable from volatile context, for contexts of 32 KB and up. - A SQLite exact-match response cache, and warm-bundle session continuity. - Fixing a context-ignore problem removed a ~23K-token directory sweep from simple first prompts. - **OpenRouter BYOK routing.** Provider-order pinning (`google–vertex`, `amazon–bedrock`) so requests bill against the client's own cloud accounts. - **10-tier model router** spanning Anthropic, OpenAI, Google, Moonshot, DeepSeek and a **$0 local tier** (Llama, Qwen Coder, LLaVA vision via Ollama). - **Observability.** A multi-terminal HUD with per-request token, cache and cost telemetry, color-coded by billing relationship. Work profiles (Quick Fix / Deep Work / Clean Slate) switch the routing strategy in one keystroke.

Why it matters for you. It gives AI-gateway capability (routing, caching, budgets, failover, vendor freedom) to any team that runs coding agents at scale, on infrastructure and billing they already own.

---

2 · AMTrix Plan: Drop-In Governance Standard for AI Coding Agents

Python (stdlib, ~23K lines, 700+ automated tests) · Claude Code hooks (adapters for Codex and Antigravity) · git hooks · 365 commits (Oct 2026) · deployed to three of my projects

Problem. AI coding agents forget between sessions, overwrite work, drift outside scope and report "done" without evidence.

What I built. One folder you drop into any repository. From then on, every AI model working there follows the same audited loop: **resume → plan → human approval → backup → edit → measured test → append-only record → handoff.**

Engineering highlights - **All seven Claude Code hook events wired** (SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, Stop, PermissionRequest, plus the stamp on every edit). Every session opens with a "where we are" block, every file is backed up before its first edit, edits to recorded history are blocked, and the hooks never block the reply. The PermissionRequest handler auto-approves only the loop's own calls under a time-boxed standing approval given in the human's words. Used live with Claude Code; adapters exist for Codex and Antigravity. - **A rules ledger** maps about 190 written obligations to the code that enforces each one (a verb, a hook, a script, or "the worker"), audited on every refresh; a

one-command cross-project health check prints one pass/fail per feature per installed project. - **A portable human-in-the-loop standard** (14 principles with do/don't/check, backup and record standards, an anti-pattern table) that a client team can adopt without the code. - **Versioned plan records.** The plans themselves live in versioned folders that lock once closed. A plan the human has already seen can only change through a recorded revision. - **Evidence written by code.** Test counts and timings are written by the `finish` step, never typed by the model. - **Generated project intelligence:** code map, file index, test-coverage map, diagrams, dashboards, catch-up reports and suggested project skills. - **Scheduled reviews** by a reviewer model. Each finding is marked *measured by code* or *reviewer judgement*. - **Safety gates.** A pre-commit secret scanner, and a hard rule: no push, install, deploy, delete, spend or model change without explicit human approval. - **Self-distribution.** One command updates every installed project and commits the update there.

Why it matters for you. It is a working reference implementation of an **agent SDLC**, with the auditability, reproducibility and rollback that change management, SOC 2 and AI-governance programs require.

---

3 · Private AI Platform: Local LLMs on Apple Silicon with Open WebUI

Ollama · Open WebUI · SearxNG · Docker Compose · Tailscale · Bash · Python · 64 GB Apple Silicon (M1 Max) workstation

Problem. Many teams can't send sensitive data to cloud LLMs, and per-token cost adds up for everyday work. Running capable models locally, reliably and with a usable interface takes real systems work.

What I built. A private AI platform on my own hardware: local model hosting plus my own chat assistant, with cloud APIs available alongside the local models when needed.

Engineering highlights - **Large models on a 64 GB unified-memory workstation:** - 30B-parameter mixture-of-experts models (Qwen3-Coder-30B-A3B, Qwen3-30B-A3B "Thinking") run as quantized GGUF builds (Q5_K_M / Q6_K) with **256K-token context as configured** (262K native). Verified on the machine with `ollama list` and `ollama show` in Oct 2026. - Llama 3.1 8B handles fast, light tasks; LLaVA 13B covers vision; nomic-embed-text provides embeddings. - **Reproducible model inventory.** Every model is declared in one config file (name, source, quantization, context length, runtime settings) and imported by script, so the setup can be rebuilt on any machine. - **Own chat assistant on Open WebUI** (Docker). One interface serves the local Ollama models and cloud model APIs (e.g., Gemini). - **Private search.** SearxNG metasearch gives answers grounded in the web without sending queries to a search vendor. - **Dual-machine architecture.** A lightweight always-on UI host, with on-demand inference on the Apple Silicon workstation over a Tailscale private network. Local and LAN access can be toggled. - **Operations:** - A one-command launcher that installs Homebrew, Ollama and Docker if missing. - Safe image updates (Watchtower) and self-healing port conflicts. - Persistent data volumes reused across rebuilds, and a maintenance script that preserves all chat history. - **Reused across the ecosystem.** The same local tier powers Claude Code Center's $0 route, AMTrix Studio and the AI Companion Toolkit.

Why it matters for you. It covers on-prem AI for privacy-sensitive data, offline capability, near-zero marginal cost, and practical guidance on which models fit which hardware before any cloud spend.

---

4 · Universal MCP Analysis Server: Multi-Model Deep-Analysis Tools for Any IDE Agent

Python · Model Context Protocol · AWS Bedrock (Converse API) · OpenRouter · built to serve my own IDE-agent workflow, then generalized to any MCP client

Problem. IDE agents choke on large codebases: file contents bloat the chat context, long answers get truncated, and follow-up questions re-read everything.

What I built. An MCP server with ten discrete tools: project AST index, deep analysis, quick summary, code review, drill-down, continue, deep research, session checkpoint, session recap and wrap-up. It is **model-agnostic** (per-call provider and model override, aliases such as Kimi, Sonnet and DeepSeek), with Bedrock as primary and OpenRouter as fallback.

Engineering highlights - **Pass-by-path.** The IDE sends only file paths and the server reads from disk, so source code never enters the agent's context. - **Anti-truncation loop.** Detects cut-off output (`finish_reason: length`) and continues automatically, up to about 60K output tokens across 8 chained calls. - **Two-stage output.** The raw exhaustive analysis is stored locally under a reference ID (7-day TTL); a compressed, schema-structured JSON summary goes back to the agent. Drill-down answers follow-ups from the stored analysis without re-processing. - **Fallback client built only on Python's standard library**, so the server keeps running when the primary provider is down. - **Embeds the Frontier Triage Engine** for audited, hashed model calls.

Why it matters for you. It is the template for connecting agents to internal code and knowledge through MCP, with the context discipline that keeps them accurate.

---

5 · Frontier Triage Engine: Audited, Reproducible, Cost-Gated Model Dispatch

Python · OpenRouter · offline test suite

- **Byte-exact requests.** Each request is serialized once and recorded with its **SHA-256 hash**; the exact same bytes are transmitted.
- **Response classification.** Every response is classed `COMPLETE / PARTIAL / FAILED / UNKNOWN`, so a truncated output is never mistaken for a finished one.
- **Audit tracer.** Monotonic attempt IDs under a file lock, atomic fsync writes, and recursive secret redaction on everything persisted.
- **Pre-flight cost card with a human y/N gate** before any paid call. Code context comes from a zero-token AST indexer.
- **Deployable into any project with one script.** The full test suite runs offline at $0.

Why it matters for you. This is model-risk discipline (*what exactly did we send, what came back, who approved the spend*) of the kind banks and insurers need for AI.

---

6 · Vision LLM Decision Agent: Real-Time Visual Perception with Deterministic Policy

Python · vision-language models · screen capture & action verification · versioned XML protocols

Problem. Vision and computer-use agents are powerful but nondeterministic. They hesitate, over-analyze and act inconsistently during long unattended sessions.

What I built. A real-time visual decision agent, applied to market-chart monitoring. A VLM reads the live screen. Decisions come from a **strict priority policy table**, and every action is verified before it counts.

Engineering highlights - **Priority lookup policy.** P0 risk override → P1 movement classification (cached at entry) → P2 stop rules → P3 counter-signal (immediate override of P2) → P4 profit protection. The model **looks up**; it doesn't deliberate. Same input, same output. - **Clean separation of perception (VLM), decision (deterministic rules) and action (verified)**, which is the architecture computer-use agents need. - **Session engineering for 4–5-hour unattended runs.** Context-window budgeting per screenshot (~1.5K tokens), session-rollover strategy, and a written feasibility and risk analysis. - **Swappable strategy cartridges, configuration audit and an action verifier.** A project generator for new vision agents went through 10+ iterations.

Why it matters for you. It automates legacy desktop and web applications with no API, along with visual QA, dashboard monitoring and screen-based triage, with predictable behavior.

---

7 · Computational Ephemeris & Multi-Factor Scoring Engine

Python · Swiss Ephemeris · pytz / geocoding · SVG rendering · strict XML schema · 9 regression & invariant test suites

Problem. Classical chart-interpretation systems (here, the Jyotish tradition) are usually done by hand or by an LLM that "guesses". The results are inconsistent and can't be verified.

What I built. A deterministic computation engine. **All mathematics happens in code. The LLM only receives validated evidence and interprets it.**

Engineering highlights - **High-precision astronomical computation.** Swiss Ephemeris planetary longitudes and speeds, Julian Day conversion, timezone normalization from geocoded birthplaces, and house-cusp computation. - **Coordinate transforms.** Tropical-to-sidereal correction with a selectable reference model (three standard ayanamsa systems), plus true lunar-node precision. - **Modular-arithmetic projections.** Eight harmonic divisional charts computed by subdividing ecliptic longitude. - **Period-timeline computation.** A 120-year planetary-period timeline derived mathematically from lunar longitude. - **Multi-factor scoring models.** A six-component weighted strength score per body, an 8×12 point-allocation matrix for transit scoring, an aspect matrix, rule-based pattern detection (15+ combinations), and a composite "net impact" score that drives visual encoding. - **Verification.** Regression tests against independently audited charts, mathematical invariants, evidence-consistency and event-integrity checks, transit-timeline matching, and **strict schema validation of every XML file sent to the LLM**.

Why it matters for you. It is the **"compute, then narrate"** pattern. The same architecture applies to credit scoring, actuarial rating, engineering calculations and any domain where an LLM must never be the one doing the math.

---

8 · Real-Time Availability Intelligence & Multi-Channel Alerting Platform

Python · SQLite · web cockpit (one framework-free HTML file) · Telegram Bot API · self-hosted ntfy with failover · Swift companion · v2 origin Oct 2025, v3 with 62 commits through Oct 2026 · runs under the AMTrix Plan standard

- **Explicit, guarded process topology.** One launch starts four daemons per product profile (feed watcher, Apple-JSON parser, HTTP dashboard, browser-session harvester). The parser and harvester are lock-elected singletons across all profiles with standby takeover; each dashboard registers a PID-verified instance; the port guard never kills another profile's server.
- **Parsed, not scraped.** Apple's fulfillment JSON is parsed into model, size, colour and store events with canonical store names and a composed direct checkout URL. Alerts follow the live dashboards: transitions for a product with no open dashboard are recorded but not pushed, and a registry failure defaults to alerting so nothing is silenced by mistake.
- **Concurrency correctness.** A cross-process file-locked alert-state store with change detection, atomic writes, schema migration, cooldowns and first-run baseline suppression, built from a root-cause analysis of duplicate and lost alerts.
- **Multi-channel dispatcher with an audit trail.** Telegram, self-hosted ntfy with primary-to-secondary failover, Discord and desktop; every delivery appends a JSONL event (provider, HTTP status, error); rich cards with three action buttons.
- **Config-driven extension.** 12 YAML product profiles over a 35-store registry with 4 anchor ZIPs; a new product line is a profile and a port; a nine-command CLI covers once/watch/sync/export/verify-links/audit-fulfillment and alert tests.
- **Telemetry into decisions.** About 9.8M observations and 60K stock transitions over six weeks analysed in SQL into per-store restock windows and median time-in-stock.
- **The production proof** that the AMTrix Plan governance standard works on a real product.

Why it matters for you. The same pattern fits inventory, pricing and SLA monitoring with alerts that reach a human within seconds.

SUPPORTING SYSTEMS

System	What it is	Stack
AMTrix Core & Skill Viewer	Metadata-routed instruction-set library: 27 skills and 7 workflow sets with YAML capability, fallback and proof fields; compilers generate a routing index, a Markdown-to-XML protocol form, editor snippets, a one-paste startup prompt and macOS/iPhone text replacements from one Markdown source; a backup-first editing CLI born from a documented data-loss incident; a serverless "cartridge" viewer whose plan-review-approve-verify board became AMTrix Plan; a QA skill wired to the gateway's live diagnostics so the model must produce HTTP payloads, SQLite rows or SSE logs as proof	Python, Bash, HTML/JS
AMTrix Signal Layer (VoiceBus + Notify-Bus)	Agents publish JSON-Schema-validated events; policy-based routing to desktop alerts, text-to-speech (per-agent voices) or a self-hosted push server; external delivery off by default. **Human-in-the-loop at a distance** for long-running agents	TypeScript, Express, ajv, chokidar, vitest, Docker Compose, Caddy
AMTrix Studio	MCP control plane: start and stop tool servers, 5 working tools, multi-provider routing (local and cloud), local vector-memory RAG	Python, Textual/Rich, YAML
PromptForge	IDE for modular, version-controlled LLM instruction sets: components, includes, conditional assembly, live build preview, diff	React, TypeScript, Vite, Google GenAI SDK
SmartBrain Voice Agents	Multi-agent voice assistant with intent routing between specialist personas, session memory, and portable deployment (Alexa Hosted / AWS Lambda / GCP Functions / local)	Node.js, Gemini
AI Companion Toolkit	Portable terminal coding assistant: user-curated context, per-call latency and token metrics, secret redaction in error paths (v1.x), MCP triage mode, command-output feedback into the prompt, git integration, formal test matrix	Python, OpenRouter, Ollama
Patch TUI	Applies only the edits declared in a JSON manifest; shows hashes and a unified diff; writes a versioned sibling file and never touches the original	Python, Bash
Mode-Scoped Agent Launcher	Explicit agent modes (discuss / research / plan / implement / verify / diagnose); a scope guard that blocks out-of-bounds edits; a 3-hypothesis debugger; evidence-stamped wrap-up; plugs into any launcher	Claude Code hooks, Node.js

System	What it is	Stack
Mermaid Dashboard Generator	Renders architecture diagrams and dashboards from structured text; SVG with tables embedded in nodes	HTML, JavaScript, Mermaid
Identity Engine	Brand definition in YAML generates a multi-format brand kit (CSS with light/dark/adaptive blocks, doc/PPT/image guidelines); a one-script drop-in theming layer with a live tuner restyles any HTML page without touching its markup	Python, YAML, CSS, JavaScript

LESSONS FROM THE TRENCHES

Every failure is logged as *friction*. The friction is reviewed and turned into a rule, and **the rule is enforced by code** rather than by memory. Friction logs run through my Agent Protocol (with a cross-project master index), AMTrix Core and AMTrix Plan, which records per-version friction and a "what we learned" report. This loop keeps me consistent and is how my expertise compounds.

What went wrong	The wisdom	Where it lives now
Lost a full working gateway build and had to recover the system	Backups are not optional, and a restore must be proven	Backup-before-first-edit hooks, versioned snapshots, git discipline in every project
AI edits silently truncated or overwrote whole files	Never let a model rewrite a whole file	Evolved from brittle diffs, to full-file replacement, to **manifest patches with hashes and versioned siblings**; net-deletion alarms
Models reported "done" when work wasn't done	A claim needs evidence	Test counts and timings written by code; reviews marked *measured* vs. *judgement*
Long instruction sets collapsed after 5–10 turns	Small beats big	Atomic skills, composed workflows, compiled context handoffs
Summaries dropped exactly the detail that mattered	Remove redundancy, not meaning	Context Compressor; raw output stored alongside every summary (MCP drill-down)
A vision agent hesitated and over-analyzed under time pressure	Where rules exist, don't let the model deliberate	Priority lookup tables; "compute, then narrate"
A provider-name casing issue caused silent "no endpoints" failures	Verify end to end with real calls	Live smoke tests; standardized error pass-through
A hidden directory sweep burned ~23K tokens on a trivial prompt	Watch what the tool actually sends	Payload capture vault, ignore policies, cost HUD
An API key appeared in an error message	Secrets leak through error paths	Recursive redaction; pre-commit secret scanning
Subscription and single-vendor lock-in	Own the routing layer	Claude Code Center; the private local tier
Forgetting my own best patterns over weeks; re-explaining projects to every new session	Memory belongs on disk, not in heads or chats	On-disk protocol, catch-up reports, automatic resume
Losses in trading, in an earlier career chapter	Risk comes first	P0 risk override in decision agents; approval gates before any spend

WHERE I FIT IN 2026

What your team is facing	What I deploy
Coding agents rolled out to hundreds of developers; spend rising, quality uneven	Gateway with caching, routing and cost telemetry, plus a governance standard in every repo
A regulated firm needs an audit trail for AI-assisted changes (model risk, SOC 2, AI governance)	Hashed request records, append-only logs, approval gates, measured evidence, backed by 15+ years in bank risk and model validation
Data that can't go to a public cloud LLM	Private model stack on local hardware, with a chat front end and private search

What your team is facing	What I deploy
Agents lose context across multi-day work	Compiled-context handoffs, catch-up reports, cross-repo protocol sync, compression
A legacy application with no API needs automation	Vision agent with a deterministic policy and verified actions
Internal knowledge and tools need to reach agents	Custom MCP servers with pass-by-path loading, anti-truncation and drill-down
Too many disconnected AI tools and scripts	Decompose, modularize and reconnect them into one plug-and-play system
Calculations that must be exact, with AI used only to explain them	A "compute, then narrate" engine with schema-validated evidence

TECHNICAL TOOLKIT

- **AI & agents:** Claude Code (hooks, skills, slash commands), Codex, Gemini CLI, Model Context Protocol (servers & clients), multi-agent orchestration, vision-language models, prompt caching, context engineering, RAG, agent evaluation and test harnesses
- **Model platforms:** Anthropic API, Google Vertex AI (incl. Model-as-a-Service), AWS Bedrock (Converse), OpenRouter (BYOK routing)
- **Local & private AI:** Ollama, Open WebUI, SearxNG, GGUF quantized models (Qwen3, Llama, LLaVA), Apple Silicon unified-memory inference, Tailscale
- **Languages:** Python, TypeScript/JavaScript (Node.js), Bash, SQL (T-SQL, PL/SQL), Rust (performance modules), Swift (components), C# / .NET
- **Back end & data:** Express, SQLite, PostgreSQL (Drizzle ORM), SQL Server, Oracle, Sybase, SSIS, data-warehouse design, JSON Schema, XML schema validation
- **Front end & visualization:** React, Vite, Radix/shadcn UI, Tailwind, Mermaid, SVG generation, terminal UIs (Textual/Rich, Bash)
- **Infrastructure:** Docker / Compose, Caddy (TLS), self-hosted push (ntfy), Telegram Bot API, GCP service-account OAuth2, git hooks, macOS and Linux automation
- **Domain:** credit risk (PD/LGD/EAD, Basel II/III IRB, CCAR, FR Y-14), model implementation testing, regulatory reporting

CAREER FOUNDATION

AMTrix Solutions Inc. · Founder & AI Systems Engineer · Apr 2022 – Present Designs and builds AI-agent infrastructure, LLM gateways, private AI platforms, MCP servers and deterministic AI pipelines (everything above). Consults on AI integration for mid-market and small-business clients.

AKM USA Trading Inc. · Principal / Technology Lead · 2017 – Apr 2022 Ran trading operations with periodic technology projects: conversational chatbots, voice/chat automation, API orchestration.

Malhotra Enterprise Solutions LLC · Lead Consultant · Nov 2013 – 2017 - Implemented wholesale PD models on a new risk-rating platform for a major regional bank. - Built a SQL Server model-implementation accuracy-testing platform and CCAR stress scenarios. - Delivered Basel II/CCAR reporting integration for a 30+ bank consortium.

Copal Amba (a Moody's Analytics company) · Vice President · May 2015 – Nov 2015 RWA calculation and forecasting architecture and FR Y-14 regulatory reporting for CCAR at a major international bank.

Moody's Analytics · Assistant Director / Lead Consultant · Jul 2007 – Nov 2013 - Led RiskAnalyst / RiskOrigin implementations for Fortune 500 financial institutions. - Built a BI platform for Basel II/CCAR model validation. - Fully automated the Credit Research Database pipeline, led its SQL Server migration, and built RiskCalc scoring integrations.

Intelligent Search Technology · Software Developer · Jun 2005 – Jul 2007 Built fuzzy name matching on the NameSearch engine, address extraction in C/C#, and SDKs for Oracle, SQL Server, .NET and PHP.

The bridge: for 15 years I validated that credit-risk models were implemented exactly as specified. I now apply the same discipline to AI agents: verify the inputs, reproduce the outputs, and leave an audit trail.

EDUCATION

Syracuse University, L.C. Smith College of Engineering and Computer Science B.S. Computer Science, Minor in Mathematics, 2005

VERIFICATION NOTE (OCT 2026)

Every project above was re-read in full in October 2026 against its code or current documentation, with one exception group. The **Private AI Platform** launcher, **PromptForge**, **SmartBrain voice agents**, **AMTrix Studio**, the **Context Compressor** and the IDE web UI are described from the earlier project write-ups only; their folders were unreadable during the re-check. The local model inventory itself (models, quantizations, context lengths) was verified directly on the machine. Facts in those six sections are kept general until they are re-read.

APPENDIX A: PROFILE COPY (LINKEDIN / UPWORK / BIO)

Headline options 1. Forward Deployed AI Engineer | Agent Reliability, LLM Gateways, Private AI & MCP | Ex-Moody's Analytics 2. I make AI agents dependable: governance, gateways, context engineering | 21 yrs enterprise tech 3. AI Agent Infrastructure Engineer | Claude Code · Vertex AI · Bedrock · Ollama · MCP | Bank-grade rigor

About (≈1,900 characters)

I build the infrastructure that makes AI agents dependable in real work.

I spent 15+ years at Moody's Analytics and in bank risk consulting, building data warehouses and validating that credit-risk models were implemented exactly as specified. Since 2025 I've applied that same rigor, full time, to LLM-agent systems.

What I've built: • A vendor-independent LLM gateway that runs the Claude Code agent on Google Vertex AI (including third-party models), AWS Bedrock, OpenRouter or local models, with caching, tool calling and cost telemetry, and no subscription lock-in. • A private AI platform: 30B-class models on a 64 GB Apple Silicon workstation with Ollama, my own Open WebUI assistant that combines local and cloud models, and private web search. • A drop-in governance standard that makes any AI coding agent resume where it stopped, back up before editing, stay in scope and prove its work with measured tests. • MCP servers that give IDE agents deep, multi-model analysis without context bloat. • Deterministic "compute, then narrate" engines, and real-time vision agents with rule-based decision policies.

How I work: I get into the details of a complex process, break it into small plug-and-play pieces, and rebuild it so it is simple to use. Everything I build is lightweight, portable and designed to connect with everything else. Every failure becomes a rule enforced by code.

I work as a forward-deployed engineer: embed with your team, find where your AI breaks, and ship the fix. Open to full-time, contract and 1099 engagements.

APPENDIX B: CLIENT INTRO (EMAIL / DM)

Hi [Name], I help teams make their AI agents dependable. Recent work: a gateway that runs Claude Code on Vertex AI, Bedrock or local models with caching and cost tracking; a private AI stack on local hardware for sensitive data; a governance layer that makes coding agents back up, stay in scope and prove their work; and custom MCP servers for deep code analysis. My background is 15+ years in bank risk technology (Moody's Analytics), so I build with audit trails by default. If your team is rolling out agents and fighting cost, context loss or trust, I'd welcome 20 minutes to compare notes.